Размер типов данных

Память на современных компьютерах, как правило, организована в блоки, которые состоят из байтов, причем каждый блок имеет свой уникальный адрес. До этого момента, память можно было сравнивать с почтовыми ящиками (с теми, которые находятся в каждом подъезде), куда мы можем поместить информацию и откуда мы её можем извлечь, а имена переменных — это всего лишь номера этих почтовых ящиков.

Тем не менее, эта аналогия не совсем подходит к программированию, так как переменные могут занимать больше 1 байта памяти. Следовательно, одна переменная может использовать 2, 4 или даже 8 последовательных адресов. Объем памяти, который использует переменная, зависит от типа данных этой переменной. Так как мы, как правило, получаем доступ к памяти через имена переменных, а не через адреса памяти, то компилятор может скрывать от нас все детали работы с переменными разных размеров.

Есть несколько причин по которым полезно знать, сколько памяти занимает определенная переменная/тип данных.

Во-первых, чем больше она занимает, тем больше информации сможет хранить. Так как каждый бит содержит либо `0`, либо `1`, то 1 бит может иметь 2 возможных значения.

2 бита могут иметь 4 возможных значения:

бит 0	бит 1
0	0
0	1
1	0
1	1

3 бита могут иметь 8 возможных значений:

бит 0	бит 1	бит 2
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

По сути, переменная с n-ным количеством бит может иметь 2^n возможных значений. Поскольку байт состоит из 8 бит, то он может иметь 2^8 (256) возможных значений.

Размер переменной накладывает ограничения на количество информации, которую она может хранить. Следовательно, переменные, которые используют больше байт, могут хранить более широкий диапазон значений.

Во-вторых, компьютеры имеют ограниченное количество свободной памяти. Каждый раз, когда мы объявляем переменную, небольшая часть этой свободной памяти выделяется до тех пор, пока переменная существует. Поскольку современные компьютеры имеют много памяти, то в большинстве случаев это не является проблемой, особенно когда в программе всего лишь несколько переменных. Тем не менее, для программ с большим количеством переменных (например, 100 000), разница между использованием 1-байтовых или 8-байтовых переменных может быть значительной.

Размер основных типов данных в C++

Возникает вопрос: «Сколько памяти занимают переменные разных типов данных?». Вы можете удивиться, но размер переменной с любым типом данных зависит от компилятора и/или архитектуры компьютера!

Язык C++ гарантирует только их минимальный размер:

	Тип	Минимальный размер
Логический тип данных	bool	1 байт
Символьный тип данных	char	1 байт
	wchar_t	1 байт
	char16_t	2 байта
	char32_t	4 байта
Целочисленный тип данных	short	2 байта
	int	2 байта
	long	4 байта
	long long	8 байт
Тип данных с плавающей запятой	float	4 байта
	double	8 байт
	long double	8 байт

Фактический размер переменных может отличаться на разных компьютерах, поэтому для его определения используют оператор sizeof.

Оператор sizeof — это унарный оператор, который вычисляет и возвращает размер определенной переменной или определенного типа данных в байтах. Вы можете скомпилировать и запустить следующую программу, чтобы выяснить, сколько занимают разные типы данных на вашем компьютере:

```
#include <iostream>

int main()
{
    std::cout << "bool:\t\t" << sizeof(bool) << " bytes" << std::endl;
    std::cout << "char:\t\t" << sizeof(char) << " bytes" << std::endl;
    std::cout << "wchar_t:\t" << sizeof(wchar_t) << " bytes" << std::endl;
    std::cout << "char16_t:\t" << sizeof(char16_t) << " bytes" << std::endl;
    std::cout << "char32_t:\t" << sizeof(char32_t) << " bytes" << std::endl;
    std::cout << "short:\t\t" << sizeof(short) << " bytes" << std::endl;
    std::cout << "int:\t\t" << sizeof(int) << " bytes" << std::endl;
    std::cout << "long:\t\t" << sizeof(long) << " bytes" << std::endl;
    std::cout << "long long:\t" << sizeof(long long) << " bytes" << std::endl;
    std::cout << "float:\t\t" << sizeof(float) << " bytes" << std::endl;
    std::cout << "double:\t\t" << sizeof(double) << " bytes" << std::endl;
    std::cout << "long double:\t" << sizeof(long double) << " bytes" << std::endl;
    return 0;
}
```

Вот результат, полученный на моем компьютере:

```
bool:           1 bytes
char:           1 bytes
wchar_t:        2 bytes
char16_t:       2 bytes
char32_t:       4 bytes
short:          2 bytes
int:            4 bytes
long:           4 bytes
long long:      8 bytes
float:          4 bytes
double:         8 bytes
long double:    8 bytes
```

Ваши результаты могут отличаться, если у вас другая архитектура, или другой компилятор. Обратите внимание, оператор sizeof не используется с **типом void**, так как последний не имеет размера.

Если вам интересно, что значит `\t` в коде, приведенном выше, то это специальный символ, который используется вместо клавиши TAB. Мы его использовали для выравнивания столбцов.

Интересно то, что sizeof — это один из трех операторов в языке C++, который является словом, а не символом (еще есть new и delete).

Вы также можете использовать оператор sizeof и с переменными:

```
#include <iostream>

int main()
{
    int x;
    std::cout << "x is " << sizeof(x) << " bytes" << std::endl;
}
```

Результат выполнения программы:

```
x is 4 bytes
```